

SQL Fundamentals

Querying, filtering, joining and managing data in relational databases

In this guide

- Introduction
- 1. Retrieving Data with SELECT
- 2. Filtering and Sorting with WHERE
- 3. Aggregation and Grouping
- 4. Joining Tables
- 5. Modifying Data and Defining Tables
- 6. Database Design, Indexes and Good Practice
- Conclusion

Introduction

SQL (Structured Query Language) is the standard language for working with relational databases. It lets you retrieve, insert, update and delete data, and it describes what result you want rather than how to compute it. Whether you use PostgreSQL, MySQL, SQL Server, SQLite or a cloud warehouse such as BigQuery, the core of SQL is remarkably consistent.

Nearly every analyst, engineer and data scientist needs SQL, because most business data lives in relational tables. This guide teaches the essentials in the order you will use them: understanding tables, selecting and filtering rows, aggregating, joining tables, modifying data and designing a schema.

The relational model in brief

- **Table:** a collection of rows and columns, like a spreadsheet with strict types.
- **Row (record):** one entry, such as a single customer.
- **Column (field):** one attribute, such as `email`, with a defined data type.
- **Primary key:** a column (or columns) that uniquely identifies each row.
- **Foreign key:** a column that references the primary key of another table, linking them.

Sample data used in this guide

Examples use two small tables. `customers(id, name, city)` stores who buys, and `orders(id, customer_id, amount, order_date, status)` stores what they buy; `orders.customer_id` is a foreign key pointing to `customers.id`.

customers.id	name	city
1	Asha	Delhi
2	Ben	London
3	Chen	Delhi

Practice tip: SQLite needs no server. Install it (or use an online sandbox) and run every example against your own sample tables.

1. Retrieving Data with SELECT

The **SELECT** statement is the heart of SQL. It chooses which columns to return, and the **FROM** clause names the table. Keywords are not case sensitive, but writing them in uppercase makes queries easier to read.

```
SELECT name, city
FROM customers;

SELECT *                -- all columns; convenient, but avoid in production
FROM customers;
```

Aliases and expressions

AS renames a column or table in the output, and you can compute new values from existing columns.

```
SELECT name AS customer_name,
       amount * 1.18 AS amount_with_tax
FROM orders;
```

DISTINCT, ORDER BY and LIMIT

- **DISTINCT** removes duplicate rows from the result.
- **ORDER BY** sorts results; add **DESC** for descending order.
- **LIMIT n** returns only the first n rows (SQL Server uses **TOP n**).

```
SELECT DISTINCT city FROM customers;

SELECT id, amount
FROM orders
ORDER BY amount DESC
LIMIT 5;                -- five largest orders
```

Logical order of execution

You write **SELECT** first, but the database processes clauses in a different order. Knowing this explains many puzzling errors, such as why an alias defined in **SELECT** cannot be used inside **WHERE**.

- 1 **FROM** and **JOIN** choose and combine tables.
- 2 **WHERE** filters individual rows.
- 3 **GROUP BY** forms groups.
- 4 **HAVING** filters groups.
- 5 **SELECT** computes output columns.
- 6 **ORDER BY** sorts and **LIMIT** trims the result.

Key points

- Select only the columns you need; it is faster and clearer.

- End each statement with a semicolon.
- Comments start with `--` or sit between `/*` and `*/`.

2. Filtering and Sorting with WHERE

The **WHERE** clause keeps only rows that satisfy a condition. Conditions combine comparison operators with **AND**, **OR** and **NOT**; use parentheses to make precedence explicit.

Operator	Meaning	Example
<code>=, <></code>	Equal, not equal	<code>status <> 'cancelled'</code>
<code>< > <= >=</code>	Comparisons	<code>amount >= 500</code>
BETWEEN	Inclusive range	<code>amount BETWEEN 100 AND 500</code>
IN	Matches any in a list	<code>city IN ('Delhi', 'London')</code>
LIKE	Pattern match (% any text, _ one char)	<code>name LIKE 'A%'</code>
IS NULL	Tests for missing values	<code>email IS NULL</code>

```
SELECT id, amount, status
FROM orders
WHERE amount > 200
      AND (status = 'shipped' OR status = 'delivered')
      AND order_date >= '2025-01-01';
```

Working with NULL

NULL represents an unknown or missing value, not zero or an empty string. Comparisons with NULL are never true, so `email = NULL` returns nothing. Always write **IS NULL** or **IS NOT NULL**. **COALESCE(x, default)** substitutes a fallback value.

```
SELECT name, COALESCE(city, 'Unknown') AS city
FROM customers
WHERE name LIKE '%an%';
```

Conditional logic with CASE

CASE works like if/else inside a query, letting you create categories on the fly.

```
SELECT id, amount,
      CASE
        WHEN amount >= 1000 THEN 'large'
        WHEN amount >= 100  THEN 'medium'
        ELSE 'small'
      END AS order_size
FROM orders;
```

Key points

- **WHERE** filters rows before grouping; **HAVING** filters after.
- Use **IS NULL**, never **= NULL**.
- Text values use single quotes; double quotes are usually for identifiers.

3. Aggregation and Grouping

Aggregate functions collapse many rows into a single value. They are the basis of reports, dashboards and summaries.

Function	Returns
COUNT(*)	Number of rows
COUNT(col)	Number of non-NULL values in a column
SUM(col)	Total
AVG(col)	Mean
MIN(col), MAX(col)	Smallest and largest value

```
SELECT COUNT(*)      AS total_orders,  
       SUM(amount)   AS revenue,  
       AVG(amount)   AS avg_order  
FROM orders;
```

GROUP BY

GROUP BY splits rows into groups and applies aggregates to each group separately. Every column in **SELECT** must either appear in **GROUP BY** or be wrapped in an aggregate.

```
SELECT customer_id,  
       COUNT(*)      AS orders,  
       SUM(amount)   AS spent  
FROM orders  
GROUP BY customer_id  
ORDER BY spent DESC;
```

Filtering groups with HAVING

Because **WHERE** runs before grouping, it cannot filter on aggregates. Use **HAVING** for that purpose.

```
SELECT customer_id, SUM(amount) AS spent  
FROM orders  
WHERE status <> 'cancelled'      -- row filter  
GROUP BY customer_id  
HAVING SUM(amount) > 1000;      -- group filter
```

Grouping by dates

Analysts often summarize by month or year. Functions differ between systems (`DATE_TRUNC` in PostgreSQL, `strftime` in SQLite, `MONTH()` in MySQL), but the pattern is the same: derive a period column and group by it.

```
-- PostgreSQL
SELECT DATE_TRUNC('month', order_date) AS month,
       SUM(amount) AS revenue
FROM orders
GROUP BY 1
ORDER BY 1;
```

Key points

- `COUNT(*)` counts rows; `COUNT(col)` ignores NULLs.
- Every non-aggregated selected column must be in `GROUP BY`.
- Use `HAVING` to filter aggregated results.

4. Joining Tables

Relational databases split information across tables to avoid repetition, so answering real questions usually requires combining them. A `JOIN` matches rows from two tables using a condition, typically equality between a foreign key and a primary key.

Types of joins

Join	Result
<code>INNER JOIN</code>	Only rows with a match in both tables
<code>LEFT JOIN</code>	All rows from the left table, plus matches from the right (NULL if none)
<code>RIGHT JOIN</code>	All rows from the right table, plus matches from the left
<code>FULL OUTER JOIN</code>	All rows from both tables, matched where possible
<code>CROSS JOIN</code>	Every combination of rows (Cartesian product)

```
-- customers with their orders (only those who ordered)
SELECT c.name, o.id AS order_id, o.amount
FROM customers AS c
INNER JOIN orders AS o
      ON o.customer_id = c.id;

-- all customers, including those with no orders
SELECT c.name, COUNT(o.id) AS order_count
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
GROUP BY c.name;
```

Finding gaps: A LEFT JOIN followed by **WHERE o.id IS NULL** returns rows that have no match, for example customers who never placed an order.

Subqueries and CTEs

A subquery is a query nested inside another. A common table expression (CTE), introduced with **WITH**, names an intermediate result and makes long queries far easier to read.

```
WITH customer_totals AS (  
    SELECT customer_id, SUM(amount) AS spent  
    FROM orders  
    GROUP BY customer_id  
)  
SELECT c.name, t.spent  
FROM customer_totals t  
JOIN customers c ON c.id = t.customer_id  
WHERE t.spent > (SELECT AVG(spent) FROM customer_totals);
```

Key points

- Always qualify columns with table aliases when joining (**c.name**).
- A missing or wrong **ON** condition can multiply rows unexpectedly.
- Prefer CTEs to deeply nested subqueries for readability.

5. Modifying Data and Defining Tables

SQL statements fall into two main groups. **DML** (Data Manipulation Language) changes rows with **INSERT**, **UPDATE** and **DELETE**. **DDL** (Data Definition Language) changes structure with **CREATE**, **ALTER** and **DROP**.

INSERT, UPDATE and DELETE

```
INSERT INTO customers (name, city)  
VALUES ('Dara', 'Mumbai'), ('Eli', 'Berlin');  
  
UPDATE orders  
SET status = 'shipped'  
WHERE id = 42;  
  
DELETE FROM orders  
WHERE status = 'cancelled' AND order_date < '2024-01-01';
```

Safety: Always include a **WHERE** clause with **UPDATE** and **DELETE**, or every row will change. Run a **SELECT** with the same condition first, and use a transaction so you can roll back.

Creating tables and choosing types

```
CREATE TABLE products (
  id          INTEGER PRIMARY KEY,
  name        VARCHAR(100) NOT NULL,
  price       DECIMAL(10, 2) CHECK (price >= 0),
  category    VARCHAR(50) DEFAULT 'general',
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Category	Common types
Whole numbers	INTEGER, BIGINT
Exact decimals (money)	DECIMAL(p, s) / NUMERIC
Text	VARCHAR(n), TEXT
Dates and times	DATE, TIMESTAMP
Logical	BOOLEAN

Constraints protect data quality

- **PRIMARY KEY** guarantees unique, non-null identifiers.
- **FOREIGN KEY ... REFERENCES** prevents links to rows that do not exist.
- **NOT NULL**, **UNIQUE**, **CHECK** and **DEFAULT** enforce rules at the database level.

Transactions

A transaction groups statements so that they succeed or fail together (atomicity). If a money transfer debits one account, the credit to the other must also happen.

```
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT;      -- or ROLLBACK; to undo both changes
```

6. Database Design, Indexes and Good Practice

Good design prevents inconsistent data and keeps queries fast. Two ideas do most of the work: normalization and indexing.

Normalization

Normalization organizes data so each fact is stored once. Suppose an orders table repeated each customer's name and city on every row. Changing a city would require updating many rows, and mistakes would create contradictions. Moving customers to their own table and referencing them by id fixes this.

- **First normal form:** each cell holds a single value; no repeating groups.

- **Second normal form:** every column depends on the whole primary key.
- **Third normal form:** columns depend only on the key, not on other non-key columns.

Analytical warehouses sometimes deliberately denormalize into wide tables to make reporting faster and simpler.

Relationships

- **One-to-many:** one customer has many orders (foreign key on the 'many' side).
- **Many-to-many:** students and courses; requires a junction table holding both keys.

Indexes

An index is a sorted structure (usually a B-tree) that lets the database find rows without scanning the whole table, much like a book's index. Indexes speed up reads on filtered or joined columns but slow down writes and use storage, so add them where queries need them.

```
CREATE INDEX idx_orders_customer ON orders (customer_id);  
  
EXPLAIN SELECT * FROM orders WHERE customer_id = 7;  -- inspect the plan
```

Query-writing best practices

- 1 Select specific columns instead of `*`.
- 2 Filter early and join on indexed keys.
- 3 Use meaningful aliases and consistent formatting.
- 4 Prefer CTEs for multi-step logic.
- 5 Test on small data, then check the plan with `EXPLAIN`.
- 6 Beware of SQL injection: never build queries by pasting user input into strings; use parameterized queries.

Conclusion

SQL provides a compact, declarative way to ask questions of structured data. You have learned how to select and filter rows, summarize with aggregates and grouping, combine tables with joins, change data safely, and design tables with keys, constraints and indexes.

Key takeaways

- Think in terms of clauses: FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY.
- Handle NULL carefully with **IS NULL** and **COALESCE**.
- Use LEFT JOIN to keep unmatched rows and INNER JOIN to keep only matches.
- Wrap related changes in transactions and always filter **UPDATE** and **DELETE** statements.
- Normalize for integrity; index for speed.

Suggested next steps

- 1 Practice on free platforms such as SQLZoo, Mode's SQL tutorial or LeetCode's database problems.
- 2 Learn window functions (**ROW_NUMBER**, **RANK**, **SUM() OVER**) for running totals and rankings.
- 3 Connect SQL to Python with **sqlite3**, SQLAlchemy or Pandas' **read_sql**.
- 4 Study query optimization and the specific features of your chosen database engine.

Remember: The best way to learn SQL is to ask real questions of real data. Pick a dataset that interests you and keep querying.